QRYVANTA Study Guide

SQL linting, ASTs, query performance and source rule reference | Edition 1.1

A practical offline course and source-aligned rule reference for Query Intelligence Studio. Rule alerts are evidence to investigate, not unconditional instructions to rewrite a query.

How to use this guide

Read the concept chapters first, then use the SQL001-SQL015 and P001-P022 sections as implementation-specific reference sheets. The rule summaries reflect the current source and its trigger conditions. Practice questions in this bundle are original, not claimed past-paper questions.

Contents

• Start here: what QRYVANTA does

• SQL fundamentals

• Parsing, dialects, and ASTs

• Static linting vs performance analysis

• Execution plans and row estimates

• Indexes and selectivity

• Joins and cardinality

• SARGability and predicates

• Query optimization workflow

• QRYVANTA architecture and data flow

• Quiz, exports, and offline use

• Practice examples and text diagrams

• SQL lint rules SQL001-SQL015

• Performance rules P001-P022

• Glossary

• Credible references

Start here: what QRYVANTA does

QRYVANTA (Query Intelligence Studio) is a desktop SQL workbench with several connected layers. The SQL Console captures complete statements; database adapters execute supported statements; SQLGlot parses SQL into a dialect-aware abstract syntax tree (AST); the linter checks code patterns; the performance analyzer combines query structure with available schema/index metadata and a normalized execution plan; the Analysis panel presents findings and recommendations; Database Explorer reads table metadata and rows.

A finding is a prompt to investigate, not proof that a query is slow. Some rules are static (they inspect syntax), while performance rules may depend on real plan and schema evidence. Confirm changes on representative data and with the target database's plan tools.

Typical workflow: connect with an authorized account -> run a representative query -> inspect lint findings -> inspect plan operations, estimated rows, indexes and predicate shape -> change one thing -> compare plans and measured elapsed time. Keep correctness and result semantics as hard requirements.

SQL fundamentals

A SELECT statement chooses a projection (columns/expressions), a source (FROM), row filters (WHERE), grouping (GROUP BY), group filters (HAVING), ordering (ORDER BY), and optional pagination (LIMIT/FETCH/OFFSET, syntax varies by dialect). SQL is declarative: you describe the result, and the optimizer chooses a physical strategy.

INSERT creates rows, UPDATE changes rows, and DELETE removes rows. An UPDATE or DELETE without a restrictive WHERE may affect every row. INSERT with an explicit column list is safer against schema/order changes. Use transactions and backups according to the database's operational policy before modifying data.

NULL means unknown or missing, not a normal value. `column = NULL` is not the test for missing values; use `IS NULL` or `IS NOT NULL`. SQL predicates use three-valued logic (TRUE, FALSE, UNKNOWN), which matters especially for NOT IN subqueries that can produce NULL.

Parsing, dialects, and ASTs

A parser turns SQL text into a structured representation. QRYVANTA uses SQLGlot and calls parse_one(query, read=dialect) before passing the tree to the linter and analyzer. The dialect matters: Oracle, PostgreSQL and MySQL differ in pagination, random functions, quoting, types and built-ins. Parsing success does not mean that the connected database will accept or execute the statement.

An abstract syntax tree (AST) represents meaning-bearing structure: SELECT, WHERE, JOIN, expressions, columns, functions, subqueries and predicates become nodes with parent/child relationships. Rules can inspect nodes instead of relying only on fragile substring matching. For example, SQL001 looks for a Star in a SELECT projection (so COUNT(*) is not the same as selecting every column), while SQL015 looks for equality/inequality nodes with a NULL node.

QRYVANTA currently has some rule-specific text checks as well as AST-based checks; this is one reason a linter finding should be reviewed in context.

Static linting vs performance analysis

Static linting asks whether the SQL shape has a known correctness, safety, maintainability or likely-efficiency risk. It can run from the parsed query and does not require a measured duration. Examples: UPDATE without WHERE, `= NULL`, leading wildcard LIKE, a large OFFSET, or LIMIT/FETCH without ORDER BY.

Performance analysis asks what access paths and operators the database plan suggests. The current implementation's normalized plan/context supports table scans, index scans, sorts, joins, temporary/materialization operations and estimated rows. It may also inspect table row counts, column types, index definitions/cardinalities and collations when the adapter provides them. Missing metadata limits what can be concluded; lack of a finding is not a guarantee of optimal performance.

The app presents a score derived from findings. Treat it as a prioritization signal, not a benchmark or a substitute for timing. Always validate changed SQL against actual result rows, permissions, concurrency and workload.

Execution plans and row estimates

An execution plan is the database's chosen physical strategy. Common operations include sequential/full table scans, index seeks/scans, nested-loop/hash/merge joins, sorts, aggregates, and temporary/intermediate results. Names vary by engine; QRYVANTA normalizes plan details into a shared model for rule checks.

Read a plan from its access paths and estimated cardinalities: how many rows enter and leave each operator? Is a selective predicate applied early? Are join keys and types compatible? Does a sort process far more rows than the caller needs? Is a table scan reasonable because the query reads much of a small table? A scan is not automatically bad; an unused index can be the correct choice for low-selectivity filters.

Estimated rows can differ from actual rows due to stale statistics, correlated predicates, skew, parameter values or unsupported estimations. Where supported, compare estimated and actual rows using the native EXPLAIN/ANALYZE tooling for your database. QRYVANTA reports available estimates and plan structure; it does not claim a wall-clock timing unless one was actually measured.

Indexes and selectivity

A B-tree index stores ordered key values and row references. It can support equality/range predicates and sometimes ORDER BY/GROUP BY, especially when a predicate uses the leading column of a composite index. An index on (department_id, created_at) does not generally give the same access path for a condition only on created_at as an index beginning with created_at.

Indexes consume storage and must be maintained by INSERT/UPDATE/DELETE. Too many or redundant indexes increase write amplification. A covering index includes the filter and projected columns needed by a query, which may reduce table lookups, but wider indexes cost more space and writes. An index on a low-cardinality column (for example, a two-value status) may be unattractive alone on a large table; the optimizer weighs selectivity, table size, caching and costs.

Before adding/dropping an index, examine the workload, constraints, leading column order, index usage and write costs. The code's P009 marks left-prefix redundancy as potential only; its recommendation explicitly asks for workload validation before removing anything.

Joins and cardinality

A join combines rows using a relationship predicate. Missing or incomplete join predicates can cause a Cartesian product or accidental many-to-many multiplication. Check key uniqueness and intended cardinalities. For each join, inspect join columns, supporting indexes, type/collation compatibility and estimated output rows.

Nested-loop joins can be excellent when an outer input is small and the inner access path is selective/indexed, but expensive when the inner side is repeatedly scanned. Hash and merge joins have different memory, ordering and input requirements. The optimizer chooses based on costs and statistics.

P006 looks for join columns without supporting leading indexes when a scanned table and join plan are present; P007 flags nested-loop plus table-scan combinations; P015 checks metadata mismatches; P017 compares join estimated output to child estimates. These are evidence-based warnings, not mandates to add indexes or rewrite every join.

SARGability and predicates

A SARGable predicate lets an index/access method search a useful key range. Wrapping an indexed column in a function, or doing arithmetic to the column, can make normal index access less usable. Example risk: `WHERE LOWER(email) = ' a@example.com'`; an alternative may normalize/store the value, compare the bare column under the right collation, or use a supported expression/function-based index.

Prefer half-open date ranges (`created_at >= :start AND created_at < :next_start`) over extracting YEAR/MONTH from every row when appropriate. Avoid implicit type conversion on indexed columns: compare a character column to a character literal, and make join-key types consistent. Keep the predicate semantically equivalent—time zones, case rules, NULL behavior and collation can change results.

The static SQL004 searches known function tokens in WHERE text. P005 inspects parsed predicate metadata for a wrapped column. P022 is narrower: it requires a wrapped predicate on an indexed column plus a table scan in the plan. Their signals have different evidence levels.

Query optimization workflow

1. Record the baseline: SQL, parameters, row count and representative data volume. 2. Confirm correctness and expected result cardinality. 3. Inspect the plan: high-row scans, repeated nested loops, large sorts, materialization, row-estimate jumps and index access. 4. Check statistics, indexes, predicate shape and join relationships. 5. Change one factor. 6. Compare estimated/actual rows and elapsed time with realistic parameters. 7. Re-test writes and concurrent workloads.

Avoid cargo-cult changes: adding an index can slow writes and may be ignored; forcing an index can harm other parameter values; replacing OR with UNION ALL can change duplicates; adding LIMIT can silently truncate results; moving predicates across an outer join can change semantics. Optimize measurable bottlenecks while preserving business logic.

QRYVANTA architecture and data flow

Main app: app.py (PySide6 window, SQL console, authentication/session UI, theme/layout switching, SQLGlot parse/analysis orchestration). Database Explorer: database_view.py. Database connectivity and profiles: database/db.py; adapter contracts and implementation: core/database.py and adapters/{oracle,mysql,postgres}.py. Parsing context and normalized plan/schema: core/query_context.py, core/context_builder.py, core/plan_model.py, core/schema_model.py. Lint rules: linter/rules.py and linter/linter.py. Performance rules and scoring/recommendations: analyzer/performance_rules.py plus analyzer/analyzer.py, score.py, recommendation.py and query_info.py. Packaging: QRYVANTA.spec and QRYVANTA-Setup.iss.

The SQL console captures complete statements and stores the history in memory for the current session. Native adapters execute supported SQL and render results; Oracle uses the external SQL*Plus executable. Saved account profiles/settings are maintained outside the program bundle, with stored passwords delegated to the OS credential store. Do not ship `.env`, saved credentials or local database files.

Quiz, exports, and offline use

LEARN, HELP, QUIZ and COMPETITIVE EXAM use local content packaged with QRYVANTA. The optional learning video is `assets/learn.mp4`; if it is missing or multimedia codecs are unavailable, the section should explain that and keep the text course usable. Developer portraits are optional and should show initials/placeholders if files are absent.

DOWNLOAD -> Database table CSV exports the rows/headers currently visible in Database Explorer. SQL history TXT exports the session's captured SQL statements, numbered and separated. Processing reports (PDF, DOCX and TXT) describe the latest analysis state, query text, available score/findings/plan text and generation timestamp; they must not invent timings or hide analysis failures.

Offline mode means these learning resources do not require a network connection. Database operations still need the relevant database server, driver, permissions and (for Oracle console) SQL*Plus. A local tutorial does not supply a database or credentials.

Practice examples and text diagrams

Projection: `SELECT customer_id, email FROM customers WHERE customer_id = 42;` selects only needed columns and can use a suitable key.

Unsafe broad write: `UPDATE accounts SET status = 'CLOSED';` affects every row unless the entire table change is intended. Add a tested predicate and use transaction safeguards.

NULL-safe test: `WHERE cancelled_at IS NULL` rather than `cancelled_at = NULL`.

Range search: `WHERE created_at >= :start_date AND created_at < :next_date` often preserves range access better than `WHERE TRUNC(created_at) = :day` (verify the dialect and index strategy).

Composite index: index (customer_id, created_at) naturally supports `WHERE customer_id = :id ORDER BY created_at`; a filter only on created_at may not use the normal leading-key seek.

Conceptual flow: SQL text -> dialect-aware parser -> AST -> static lint checks -> query/schema/plan context -> performance rules -> findings + recommendations + score -> analyst validates plan and measurements.

Conceptual join check: rows from A + rows from B --(relationship predicate)--> joined rows. If the predicate is missing, A x B can multiply row counts. A large estimated increase deserves investigation, not automatic rejection.

SQL lint rules SQL001-SQL015

These summaries follow the current implementation in linter/rules.py. A rule may use syntax-oriented heuristics and can have false positives or false negatives on uncommon dialect syntax.

SQL001 - SELECT * [Medium]

Code trigger: The parsed SELECT projection contains a bare Star node; aggregate COUNT(*) is not what this rule is targeting.

Why it matters: Selecting every column can transfer unnecessary data and make code more sensitive to schema changes.

Recommended response: Name the columns the application needs and validate that the result shape remains correct.

SQL002 - UPDATE without WHERE [High]

Code trigger: The statement contains an UPDATE node and no WHERE node in the parsed tree.

Why it matters: Without a restrictive condition, an update can modify every row in the target table.

Recommended response: Add and test the intended predicate; use transaction/rollback safeguards for important writes.

SQL003 - DELETE without WHERE [High]

Code trigger: The statement contains a DELETE node and no WHERE node in the parsed tree.

Why it matters: Without a condition, all rows in the target table can be deleted.

Recommended response: Add and verify a restrictive WHERE clause, preview matching rows, and use appropriate transaction safeguards.

SQL004 - Non-SARGable WHERE function [Medium]

Code trigger: The implementation searches WHERE text for a set of functions including YEAR, MONTH, DAY, UPPER, LOWER, DATE, CAST, TRUNC, TO_CHAR, TO_DATE, TO_NUMBER and NVL.

Why it matters: Applying a function to a column may prevent a normal index from serving the predicate efficiently.

Recommended response: When semantically valid, rewrite the predicate to compare a bare column to a transformed value; otherwise evaluate an expression index.

SQL005 - SELECT without WHERE [Low]

Code trigger: The rule finds a SELECT without any WHERE, but suppresses the finding for aggregate functions COUNT/SUM/AVG/MIN/MAX and GROUP BY.

Why it matters: The query may read a broad set of rows; intentional reporting scans are possible.

Recommended response: Add a selective condition when the application does not need all rows; judge intentional aggregates/reporting separately.

SQL006 - Leading wildcard LIKE [Medium]

Code trigger: The LIKE pattern is a string literal whose value starts with `%` or `_`.

Why it matters: A leading wildcard prevents a normal B-tree index from efficiently seeking to the beginning of a key range.

Recommended response: Prefer prefix matching such as `name LIKE 'abc%'` when valid, or choose full-text/search functionality.

SQL007 - NOT IN with subquery [High]

Code trigger: A NOT node wraps an IN node that contains a query/subquery.

Why it matters: If the subquery returns NULL, three-valued logic can make NOT IN return unexpected results.

Recommended response: Consider NOT EXISTS or an equivalent anti-join and check NULL semantics and the execution plan.

SQL008 - Large OFFSET [Medium]

Code trigger: The top SELECT has a literal integer OFFSET of at least 1000.

Why it matters: Large offsets may require processing and discarding many rows before returning a page.

Recommended response: Consider keyset/seek pagination with a stable, indexed sort key.

SQL009 - Limited result without ORDER BY [Low]

Code trigger: The top SELECT has a limit node and no order node; a literal LIMIT 1 is deliberately suppressed by the source rule.

Why it matters: Without a defined order, the limited subset is not deterministic and can change across plans or runs.

Recommended response: Add a stable ORDER BY that reflects the intended page order, ideally with a unique tie-breaker.

SQL010 - JOIN without condition / comma-join syntax [Varies: Low/Medium/High]

Code trigger: An explicit CROSS JOIN is reported at Medium severity; a JOIN lacking ON/USING and multiple FROM sources without a connecting predicate are High; comma-join syntax whose WHERE has a cross-table column equality is Low.

Why it matters: Missing relationships can create Cartesian row multiplication; legacy comma-join syntax can also be harder to review.

Recommended response: Add ON/USING or a correct relationship predicate; prefer explicit JOIN ... ON syntax. Keep CROSS JOIN only when the Cartesian result is intentional.

SQL011 - INSERT without column list [Medium]

Code trigger: An INSERT whose target is a Table rather than an explicit Schema column target is flagged.

Why it matters: Values then depend on positional table order, making schema changes risky.

Recommended response: Use `INSERT INTO table (col1, col2) VALUES (...)` and provide matching values.

SQL012 - OR across different columns [Medium]

Code trigger: The WHERE AST contains an OR whose left/right branches refer to different sets of column names.

Why it matters: The different branches may need different access paths and can be harder to optimize.

Recommended response: Review selectivity; consider equivalent UNION ALL branches only if duplicate/result semantics are preserved, then compare plans.

SQL013 - Random ordering [High]

Code trigger: ORDER BY expressions are checked for dialect-specific random functions, including RAND, RANDOM, DBMS_RANDOM and NEWID; Oracle includes extra DBMS_RANDOM AST/text checks.

Why it matters: Random values commonly need to be evaluated for many rows and then sorted.

Recommended response: Use a sampling strategy suitable for the database and requirement; validate distribution and result semantics.

SQL014 - Scalar subquery in SELECT list [Medium]

Code trigger: A top-level SELECT projection expression contains a Subquery node.

Why it matters: A scalar subquery may add repeated work per outer row, especially if correlated.

Recommended response: Consider a JOIN, pre-aggregated derived table or window expression when equivalent; verify duplicate and NULL behavior.

SQL015 - Equality comparison to NULL [High]

Code trigger: An AST equality or inequality node has a NULL on either side (`= NULL` or `<> NULL`).

Why it matters: NULL comparisons evaluate to UNKNOWN rather than TRUE/FALSE in the expected way.

Recommended response: Use `IS NULL` or `IS NOT NULL`.

Performance rules P001-P022

These summaries follow analyzer/performance_rules.py and the normalized plan/context data it reads. Some checks require plan nodes or schema metadata. Source thresholds include LARGE_ROWS=10000, INDEX_FRACTION_THRESHOLD=0.30, ROW_EXPLOSION_FACTOR=2.0, LOW_CARDINALITY_MAX_DISTINCT=10, COVERING_MIN_ROWS=1000 and COVERING_MAX_COLUMNS=4.

P001 - Full table scan [High]

Code trigger: A plan table-scan operation is detected by the normalized query context.

Why it matters: A scan reads a table's blocks rather than finding rows through an index; this can be expensive on a large input, but optimal for small or low-selectivity reads.

Recommended response: Check table size, predicate selectivity, statistics and required output before choosing an alternate access path.

P002 - Filtered scan without supporting index [High]

Code trigger: For a bare filter predicate, the target table is scanned and the filtered column has no leading supporting index.

Why it matters: A selective filter may benefit from index access, depending on table size and selectivity.

Recommended response: Evaluate a suitable index and validate read improvement against write/storage cost.

P003 - Explicit sort operation [Medium]

Code trigger: The normalized plan contains a sort operation.

Why it matters: Sorting costs CPU and memory and can spill to temporary storage for large inputs.

Recommended response: Check whether a suitable index can supply the requested ORDER BY/GROUP BY order; verify the plan.

P004 - Large unrestricted scan [High]

Code trigger: The statement has no WHERE and a table scan has estimated rows or known table row count at least the configured LARGE_ROWS threshold.

Why it matters: An unrestricted read from a large table can move many rows and consume resources.

Recommended response: Restrict the result if the application does not require the entire table; do not add a filter that changes intended output.

P005 - Non-SARGable filter [Medium]

Code trigger: A predicate in the query context is marked wrapped (function/arithmetic around a column).

Why it matters: The normal index key may not be usable as a seek for the transformed expression.

Recommended response: Rewrite the predicate to leave the column bare where valid, or use an expression index if supported.

P006 - Join column without supporting index [Medium]

Code trigger: Join pairs and join plan exist; a scanned join table has no detected index on its join column.

Why it matters: Repeated lookup or join processing may be expensive without a useful access path.

Recommended response: Consider an index only after checking selectivity, join direction and write overhead.

P007 - Nested loop with table scan [Medium]

Code trigger: The plan has a join plus table scans and includes a NESTED_LOOPS or NESTED_LOOP_JOIN operation.

Why it matters: The inner/scanned side may be revisited many times, magnifying scan cost.

Recommended response: Review row estimates and possible indexed access; nested loops are not inherently bad when the outer side is small.

P008 - Implicit conversion on string column [Medium]

Code trigger: A bare comparison predicate uses a numeric literal against a column whose metadata type is in the code's STRING_TYPES set.

Why it matters: The database may convert a value or column and make comparisons/index usage less predictable.

Recommended response: Use a type-compatible literal or parameter unless conversion is intentional; re-check application semantics.

P009 - Potentially redundant index [Low]

Code trigger: A non-primary, non-unique index's columns are a left-prefix of another index's column list.

Why it matters: The longer index may support the shorter index's access pattern, but constraints and workload can justify both.

Recommended response: Validate constraints, measured usage, storage and write cost before dropping the shorter index.

P010 - Composite index leading column missed [Medium]

Code trigger: The query filters a non-leading column of a composite index but not its first column.

Why it matters: A normal B-tree seek generally starts from the leading key, so suffix-only filtering can be less effective.

Recommended response: Add the leading predicate only if logically correct, or design an index whose leading columns match workload.

P011 - Covering-index opportunity [Low]

Code trigger: The code excludes SELECT-star, respects a small-table threshold, and looks for a small selected/filter column set not already covered by an index.

Why it matters: A covering index may avoid table lookups for a query that needs few columns, but increases index size/write work.

Recommended response: Test a narrow candidate index against read benefits, storage and write overhead.

P012 - GROUP BY lacks leading index [Medium]

Code trigger: The query has GROUP BY columns without an index leading on a grouped column and the plan shows a sort/grouping operation.

Why it matters: The engine may need an additional sort or grouping step.

Recommended response: Consider a matching index when the measured workload justifies its cost.

P013 - Heavy column without LIMIT [Medium]

Code trigger: A selected column's metadata type is in the code's HEAVY_TYPES set and the query has no LIMIT/FETCH limit.

Why it matters: Large text/binary or similar values can increase I/O, memory and result-transfer cost.

Recommended response: Fetch heavy columns only when needed, paginate metadata, or retrieve large payloads separately.

P014 - UPDATE changes indexed column [Medium]

Code trigger: An UPDATE assignment changes a column that the table's known index metadata says is indexed.

Why it matters: The database must maintain index entries when indexed values change, increasing write work.

Recommended response: Keep indexes that serve workload; avoid unnecessary indexes on frequently changed columns.

P015 - Join metadata type/collation mismatch [High]

Code trigger: Join-column metadata differs in data type or (when both present) collation.

Why it matters: Conversions or incompatible comparison semantics may hinder index matching and can cause correctness surprises.

Recommended response: Align definitions where appropriate and inspect the native execution plan.

P016 - Index exists but is not used [Medium]

Code trigger: A filtered column has a leading index and the normalized plan still scans that table.

Why it matters: The optimizer may reject the index due to selectivity, statistics, conversion, or cost.

Recommended response: Inspect statistics, selectivity and metadata before adding a second index or forcing a plan.

P017 - Join row explosion [High]

Code trigger: For plans with joins and at least two child row estimates, join output estimate meets the code's LARGE_ROWS / ROW_EXPLOSION_FACTOR threshold relative to largest child.

Why it matters: A very large join result relative to its inputs can indicate many-to-many multiplication or missing predicates.

Recommended response: Verify join keys, uniqueness/cardinality and intended relationship; reduce rows only when semantics allow.

P018 - Large sort without limit [High]

Code trigger: No query limit is present and a sort operation estimates rows at least LARGE_ROWS threshold.

Why it matters: Sorting a large result without limiting rows can use substantial CPU/memory/temp I/O.

Recommended response: Limit only when application semantics allow, or test an index-backed order.

P019 - Temporary/materialized intermediate [Medium]

Code trigger: The normalized plan reports a temporary operation/materialization.

Why it matters: An intermediate result may require extra memory, disk I/O and CPU.

Recommended response: Inspect GROUP BY, DISTINCT, ORDER BY, CTEs and derived tables; simplify only if result semantics are preserved.

P020 - Large fraction read through index [Medium]

Code trigger: For an index scan with row estimates and known total table rows, estimated rows/table rows is at least 30%.

Why it matters: Random index access may be less efficient than a scan when a large fraction of rows is needed.

Recommended response: Compare plan costs and actual performance; do not force an index simply because it exists.

P021 - Low-cardinality leading index [Medium]

Code trigger: The query has a WHERE equality predicate on a large table and a leading index cardinality is known and at or below the configured low-cardinality distinct-value limit.

Why it matters: A low-selectivity key can identify many rows, making the index alone less useful.

Recommended response: Test a more selective composite index or let the optimizer scan if that is cheaper.

P022 - Indexed wrapped predicate with table scan [High]

Code trigger: A WHERE predicate is wrapped, that column has an index, and the plan contains a table scan.

Why it matters: The function/expression appears associated with the indexed column not being used for the access path.

Recommended response: Rewrite to a bare-column predicate when equivalent or consider a database-supported expression index.

Glossary

AST: Abstract syntax tree: a structured representation of SQL statements produced by the parser.

Cardinality: The number of rows expected or observed at an operator or result.

Composite index: An index whose key contains multiple columns in a defined order.

Covering index: An index containing all columns needed for a query path, potentially avoiding table lookups.

Dialect: A database's SQL grammar and behavior variant, such as Oracle, PostgreSQL or MySQL.

Execution plan: The physical operators and access paths selected by the database optimizer.

Index selectivity: How narrowly an index/predicate identifies rows; selective predicates match a small fraction.

Keyset pagination: Paging by a stable key boundary rather than skipping a large OFFSET count.

Materialization: Producing an intermediate result, often into temporary memory or storage, for later operators.

NULL: A marker for unknown/missing information; it is not equal to a normal value, including another NULL.

Nested loop: A join strategy that iterates one input and probes the other input for each row or batch.

Optimizer: The database component that estimates alternatives and chooses a plan.

SARGable: A predicate form that can efficiently use an index/search access path.

Statistics: Metadata such as row counts and value distributions used by the optimizer for estimates.

Table scan: Reading table rows/blocks without using an index as the primary row access path.

Three-valued logic: SQL predicate results may be TRUE, FALSE or UNKNOWN, especially with NULL.

Credible references

• **SQLGlot documentation** — Parser, AST and dialect-aware SQL transpilation. https://sqlglot.com/

• **PostgreSQL: Using EXPLAIN** — Official explanation of plan nodes, costs and row estimates. https://www.postgresql.org/docs/current/using-explain.html

• **Oracle Database SQL Tuning Guide** — Official Oracle optimizer and SQL tuning documentation; select the guide version matching your database. https://docs.oracle.com/en/database/oracle/oracle-database/

• **MySQL Reference Manual: EXPLAIN** — Official MySQL execution-plan documentation. https://dev.mysql.com/doc/refman/8.4/en/explain.html

• **SQLite Query Planning** — Accessible explanation of indexes and query planning. https://www.sqlite.org/queryplanner.html

• **PostgreSQL: Indexes** — Official guide to index types and usage. https://www.postgresql.org/docs/current/indexes.html

• **OWASP SQL Injection Prevention Cheat Sheet** — Practical guidance on safe parameterized SQL; relevant when building apps around queries. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html